

Examen réparti 2 DataCloud

Master 2 SAR

Partie Pratique

Jonathan Lejeune

Février 2025

2 heures

Tout document papier ou électronique autorisé
Tout moyen de communication ou d'accès à internet interdit

URL des ressources

<https://www-licence.ufr-info-p6.jussieu.fr/lmd/licence/2024/ue/MU5IN451-2024oct/>

Consignes générales

- Le barème est sur 12 points et est donné à titre indicatif.
- Votre rendu sera corrigé automatiquement, respectez-bien les noms de classe et de package demandés.
- Votre session est en mode examen : elle est vierge de tout fichier et vous n'avez pas accès au web.
- Lorsque vous souhaitez rendre votre copie, il suffit de vous déconnecter et de quitter la salle, **aucun rendu Moodle n'est à faire**
- Ce travail est à faire **seul**.
- Le sujet est composé de 5 exercices indépendants. Il n'est pas nécessaire de réussir un exercice pour en réaliser un autre
- Les tests JUnit compileront au fur et à mesure de votre avancement dans le sujet. Il vous est donc conseillé de les intégrer dans votre projet progressivement (valable surtout pour Eclipse)
- Votre note dépendra du nombre de tests passés avec succès qui vous rapportera un certain nombre de points selon le barème donné à titre indicatif.
- Un fichier source qui n'est pas trouvé ou qui ne se trouve pas au bon endroit ou qui ne compile pas ne rapporte aucun point.
- Votre rendu sera le dossier racine de votre projet scala et qui doit contenir l'arborescence de vos fichiers sources.

Mise en place des exercices 1 à 3

1. Démarrer un ide scala. Vous avez à votre disposition :
 - eclipse avec plugin scala (IDE de référence) accessible par la commande `eclipse-scala`.
À l'ouverture penser à passer en perspective scala
 - intelliJ (avec plugin scala) accessible par la commande `idea.sh`

2. Créer un projet scala nommé *copie-NOM* en remplaçant *NOM* par votre nom de famille et y ajouter les librairies suivantes :
 - Junit 4 (intégré dans l'IDE normalement)
 - *datagenerator.jar* et *datageneratorext_hbaseext.jar* disponibles sur la page web de l'examen
 - les fichiers jar présents dans le répertoire `/usr/DataCloud/dc/spark-3.5.0-bin-hadoop3/jars`
3. Créer deux source folders : un pour les ressources, un pour votre rendu
4. Intégrer correctement les ressources fournies dans le source folder correspondant
5. Dans le source folder de votre rendu, y créer un package `datacloud.exam2025` et y déclarer un objet singleton scala `Copie`.

Important :

- **Votre rendu se résumera uniquement au fichier *datacloud/exam2025/Copie.scala*.** C'est dans ce fichier que vous répondrez à tous les exercices du sujet, **assurez-vous donc qu'il compile avant de rendre votre copie**
- **Il est formellement interdit d'utiliser les méthodes `collect` ou `take` du RDD pour réaliser les fonctions demandées**

Exercice 1

Nous reprenons l'exemple du moteur de recherche Noodle vu en TD. Pour rappel Noodle est un moteur de recherche de pages web. À chaque nouvelle requête reçue, une ligne de log est sauvegardée. Cette ligne de log est formatée de la manière suivante :

- le premier mot indique la date de la connexion au format `JJ_MM_AAAA_HH_MM_SS`
- le deuxième mot indique l'adresse IP du client ayant fait la requête
- le reste de la ligne contient les mots-clés de la requête. Chaque mot-clé est séparé par le caractère `+`.

Nous donnons comme exemple la ligne ci-dessous où les informations sont séparées par un espace :

```
02_11_2012_12_32_10 132.227.045.028 musique+orientale
```

Ceci indique que le client 132.227.45.28 a fait une requête comportant les mots clés « musique » et « orientale » le 02/11/2012 à 12h 32min 10sec.

Question 1

Dans *Copie.scala*, programmer la méthode `exo1(rddin:RDD[String],jour:Int, mois:Int,annee:Int):RDD[String]` qui permet de **lister les adresses IP distinctes (pas de doublon) qui ont fait une requête pour un jour donné :**

- `rddin` est le rdd contenant les lignes de log respectant le format décrit ci-dessus
- `jour`, `mois` et `annee` désigne la date du jour en question
- la méthode renvoie un RDD qui contient sans doublons les adresses IP (sous le format d'une chaîne de caractères) du jour en question

Validez votre code avec le test `datacloud.exam2025.test.TestExo1` (2 pts)

Exercice 2

Question 1

Dans le même contexte de l'exercice précédent, ajouter dans le fichier *Copie.scala* la méthode `exo2(rddin:RDD[String]):RDD[(Int,(String,Int))]` qui calcule **le nombre de requêtes reçues et le mot le plus recherché par tranche horaire d'une heure :**

- `rddin` est le rdd contenant les lignes de log respectant le format de log

- la méthode renvoie un RDD de couples dont la partie de gauche est un entier égal à une heure journalière (compris donc entre 0 et 23) désignant ainsi la tranche horaire et la partie de droite est un couple associant la mot le plus recherché (partie de gauche) et le nombre de requêtes (partie de droite) de cette tranche horaire.

N.B. : Si vous devez appeler la méthode `String::split` avec le séparateur `+` (qui est un caractère spécial pour les regexp), il est nécessaire d'appeler la méthode ainsi : `s.split("\\+")`

Validez votre code avec le test `datacloud.exam2025.test.TestExo2` (2 pts)

Exercice 3

Nous reprenons l'exemple de l'entreprise StereoPrix vu en TD qui génère une ligne de log pour chaque produit vendu. Une ligne de log contient les informations suivantes séparées par un espace :

- la date et l'heure de vente au format `JJ_MM_AAAA_HH_MM_SS`
- le nom du magasin où le produit a été vendu
- le prix de vente (valeur entière)
- la dénomination du produit
- la catégorie du produit (ex : fruits et légumes, électroménager, jouet,)

Voici un exemple de ligne

```
03_02_2025_10_52_26 paris5 150 aspirateur electromenager
```

Question 1

Dans *Copie.scala*, programmer la méthode `exo3(rddin:RDD[String]):String` qui, pour un RDD de logs passé en paramètre renvoie le nom du magasin ayant **le plus grand chiffre d'affaires journalier moyen**. Il est donc nécessaire de calculer :

- 1) pour chaque magasin son chiffre d'affaires pour chaque jour (=le montant total de toutes ses ventes par jour)
- 2) puis pour chaque magasin le chiffre d'affaires journalier moyen
- 3) puis extraire le magasin ayant la valeur la plus élevée

Validez votre code avec le test `datacloud.exam2025.test.TestExo3` (3 pts)

Mise en place des exercices 4 et 5

Compléter la configuration actuelle ainsi

1. Ouvrir un terminal et configurer les clés ssh :
 - (a) Créer une paire de clés ssh : `ssh-keygen -q -N '' -f ${HOME}/.ssh/id_rsa`
 - (b) Ajouter la clé publique aux clés autorisées : `cat ${HOME}/.ssh/id_rsa.pub >> ${HOME}/.ssh/authorized_keys`
 - (c) Taper `ssh localhost` et vérifier qu'aucun mot de passe ne soit demandé.
 - (d) Taper `exit`
2. Ouvrir un navigateur et taper l'URL indiquée au début du document
3. Télécharger le script `env.sh`. À chaque nouvelle fenêtre de terminal que vous ouvrez, il faudra exécuter cette commande via `source` (taper `source /chemin/vers/env.sh`)
4. Ouvrir un terminal (+ étape précédente) et formater un nouvel HDFS : `hdfs namenode -format`
5. Démarrer le HDFS : `start-dfs.sh`. Vérifier son bon démarrage avec le script `check_start.sh` disponible sur la page web de l'examen (penser à lui donner les droits en exécution avec la commande `chmod +x`). Il est possible que `secondaryNameNode` ne démarre pas ce qui est sans importance.

6. Démarrer Hbase : `start-hbase.sh`. Vérifier son bon démarrage avec le script `check_start.sh`.
7. Dans votre projet scala nommé ajouter les jars présents dans les répertoires :
 - `/usr/DataCloud/dc/hbase-2.4.7/lib`
 - `/usr/DataCloud/dc/hbase-2.4.7/lib/client-facing-thirdparty`

Exercice 4

Nous nous situons dans le contexte du site web de radio en ligne et de musique communautaire LastFM. Son système logue dans des fichiers textes pour chaque utilisateur et pour chaque titre le nombre de fois où l'utilisateur a écouté le titre en local, le nombre de fois où l'utilisateur a écouté le titre en ligne et le nombre de fois où l'utilisateur l'a passé sans l'écouter. Chaque champ est séparé par un espace. Une ligne de log a donc ce format.

```
userid trackid locallistening radiolistening skip
```

Le but est de remplir avec Spark, à partir des données textuelles des fichiers, une table Hbase ayant le schéma suivant :

rowkey	compteurs				
useridtrackid	userid	trackid	locallistening	radiolistening	skip

La table possédera une seule column family (**compteurs**). Le rowkey de cette table sera une concaténation du **userId** avec le **trackId**. Les fichiers n'assurant pas l'unicité de la clé (il peut y avoir plusieurs lignes ayant les mêmes **userid** et **trackid**), il est donc nécessaire d'incrémenter les compteurs d'une ligne existante et non de les écraser.

Question 1

Dans *Copie.scala*, en utilisant l'API des RDD de Spark et le fichier *SparkHbaseConnector.scala* (vu en TME et fourni dans les ressources), programmer la méthode `exo4(rddin:RDD[String], dest:String):Unit` qui permet de faire le remplissage de la table. Elle prend en paramètre un RDD contenant les lignes de log des fichiers d'entrée et le nom de la table destination. La table sera supposée déjà créée avant l'appel à cette méthode.

Validez votre code avec le test `datacloud.exam2025.test.TestExo4` (3 pts)

Exercice 5

Nous souhaitons créer une nouvelle table à partir d'une réorganisation des données de la table créée à l'exercice précédent. Le but est d'avoir une table ayant pour rowkey les **trackid** et une colonne par association d'un type de compteurs (local,radio et skip) avec un **userid**. Le nom de la colonne sera la concaténation des deux informations (type du compteurs et userid) séparées par un underscore. On gardera qu'une seule column family de nom **compteurs**. Le schéma de la nouvelle table est donc le suivant :

rowkey	compteurs						
trackid	locallistening_userX	radiolistening_userX	skip_userX	locallistening_userY	radiolistening_userY	skip_userY	...

Question 1

Dans *Copie.scala*, en utilisant l'API des RDD de Spark et le fichier *SparkHbaseConnector.scala*, programmer la méthode `exo5(listening:String,dest:String,sc:SparkContext):Unit` qui permet de faire cette réorganisation :

- **listening** désigne la table initiale créée à l'exercice précédent
- **dest** désigne la table destination, i.e. la nouvelle table (supposée créée) réorganisant les données
- **sc** le contexte Spark permettant de charger la table source en RDD.

Validez votre code avec le test `datacloud.exam2025.test.TestExo5` (2 pts)